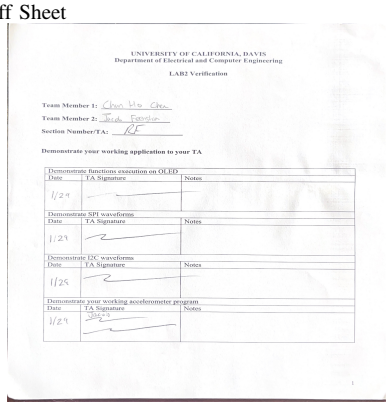
Serial Interfacing with SPI and I2C

Jacob Feenstra and Chun-Ho Chen

I. Checkoff

Fig. 1. Checkoff Sheet



UNIVERSITY OF CALIFORNIA, DAVIS
Department of Electrical and Computer Engineering

LAB2 Verification

Team Member 1: Chun Ho Chen
Team Member 2: Jacob Feenstra
Section Number/TA: RF

Demonstrate your working application to your TA

Demonstrate functions execution on OLED		
Date	TA Signature	Notes
1/29		
Demonstrate SPI waveforms		
Date	TA Signature	Notes
1/29		
Demonstrate I2C waveforms		
Date	TA Signature	Notes
1/29		
Demonstrate your working accelerometer program		
Date	TA Signature	Notes
1/29		

II. Introduction

In this lab, we familiarized ourselves with using communication protocols (SPI and I2C, respectively) to communicate to peripheral devices with our CC3200 Microntroller Unit (MCU).

1) We used the SPI protocol to interact with an Adafruit SSD1351 OLED Digital Display (OLED), and perform drawing operations on it's screen. Using the general-purpose pins provided by the MCU, we can bind the signals defined by the SPI protocol and initiate communication via the SPI software development kit API.
2) Then, we introduce I2C to extract acceleration data in 3 dimensions, from three registers defined by the MCU's on-board BMA222 Accelerometer. We focus on the x and y dimensions, determining the tilt & angle of the MCU in 8-bit 2's complement form, from two respective registers.
3) Combining our ability to communicate with the OLED and determine accelerometer information, we can tilt the accelerometer and get a velocity response from drawn pixels on the OLED. Mainly, we move a circle around the OLED display. More information on how this works will be included in subsequent sections.

In completing this lab, we effectively learn how to work with MCU-supported communication protocols, and get it to do exciting things with peripheral devices, as well as utilize on-board MCU components to introduce additional functionality to these peripheral device programs that we otherwise wouldn't have.

III. Background

Modules, hardware, and software tools used are as follows.

A. Modules

1) `spi_demo`: Texas Instruments CCS project that demonstrates how to use SPI signals for simple UART communication. This module is co-opted and used for all parts of the lab when sending data to the OLED. Note this also includes usage for the MCU as both a master & slave: we leave just the code for master, since that is all that the MCU will be doing.
2) `i2c_demo`: Texas Instruments CCS project that demonstrates how to use I2C signals for simple reading operations with the BMA222 accelerometer. Namely, using the UART terminal to print out the changing accelerometer axis values (x, y, and z) when the MCU is rotated in physical space.
3) `Adafruit SSD1351 Open Source Graphics Library`: Abridged from the Adafruit graphics library, provides an API to write commands (and their subsequent data) to the OLED, as well as turn it on. Includes various functions to draw complex shapes on the digital display. This library is used extensively in this lab & modified. More information on the modifications will be detailed.

B. Hardware

1) `CC3200 LaunchPad XL`: The microcontroller unit that orchestrates the SPI and I2C protocols in this lab.
2) `Adafruit SSD1351 OLED Display`: Digital display that accepts written commands and data from communication protocols, and prints pixel output.
3) `Breadboard`: The mount for the OLED, and for quick and easy configuration of pin connectors to the OLED (from the MCU).
4) `BMA222 Accelerometer`: On-board accelerometer that records the orientation of the MCU.
5) `Saleae Logic Analyzer`: Logic Analyzer to read MCU pin signals across 4 channels. Allows for graphical display of a signal when coupled with software.

C. Software

1) `TI CCS Studio`: Integrated Development Environment to work with the MCU, debug it, and write C code.
2) `TI SysConfig`: Software to multiplex pins for configuration of pinouts.
3) `Saleae Logic 2`: GUI software to display signals from the Saleae Logic Analyzer across 4 channels, and interpret data based off analyzer profiles.

IV. Goals

There are four checkoff tasks to complete associated with this lab section.

A. Implement OLED Interface Using SPI

- Arguably the most involved checkoff in this lab, this involves setting up the SPI communication protocol to write commands and their subsequent data to the OLED display, and implementing the API provided by Adafruit to draw various shapes on the display (as well as producing the coding framework needed to use some of these API calls effectively).
- One need to carefully set up the SPI pins, and then use the tools which the SDK provides to create functions that allow for the OLED API to be implemented; namely completing `writeCommand()` and `writeData()`. This happens in `Adafruit_OLED.c`. More on this in Methods!
- Turning our attention to `main.c`, the systems programmer needs to ensure the SPI is configured properly; this ended up being quite touchy, and required careful consideration of the values that we could edit.
- Lastly, a main `while(1)` loop needs to be set up to invoke the appropriate drawing functions.
- The main lessons from this checkoff consists of working around the software inconviences of Texas Instruments SDK's and open-source firmware. Getting this program working requires a careful touch, which will be detailed in methods. This lab works as a good primer for the kind of attention to detail required of embedded systems design. It also helps to explore the low-level requirements of working with peripheral devices like the OLED, and the kind of necessities needed for supporting the functionality the programmer desires, both in terms of software and hardware.

B. Verifying SPI waveforms using a Saleae logic

- This lab introduces the Saleae Logic Analyzer, and it's corresponding software Saleae Logic 2. This is an important debugging tool in the arsenal of an embedded systems programmer, allowing for easy verification of signal traces for up to 4 pin-outs (organized by 4 channels in the Saleae Logic 2).
- Checkoff 2 also helps understand the analog component underpinning the software we have written, and contextualizes not only what our software is doing, but how the SPI protocol is working. We get a feel for what a serial clock should look like, what the MOSI (data line) should look like for the OLED, as well as how the DC, Reset, and CS pins shThould behave. The analyzer provides an incredibly powerful snapshot of all of this working in concert, and how different systems can communicte with each other.

C. Controlling the BMA222 accelerometer with I2C

- In this checkoff, we get familiar with using the on-board BMA222 accelerometer to determine the orientation of the MCU, and eventually, use it's output to produce interesting results with peripheral devices.
- The checkoff consists of running the `i2c_demo`, and verifying the output from the registers defined by the BMA222 (indicating the tilt of the MCU). The SDA and SDL pins are used to interact with the I2C protocol, and we use the UART protocol to type commands in the terminal to read the values of specific registers (indicating the accelerometer axis values at the time of printout).

D. Sliding Ball

- In Checkoff 4, we bring it all together, combining the BMA222 with the OLED, enabling a ball to move on the screen.
- This checkoff requires that you extract the relevant values from the BMA222 registers (x and y), and use them in a formula to simulate a "natural acceleration" increase for the central coordinate points of a small circle on the OLED display. It requires careful consideration of how these values are used as scalar multiples; their 2's complement nature also needs to be carefully handled (signed versus unsigned integers). Ultimately, this checkoff teaches careful data type application and conditional control of math operations in the context of embedded systems, allowing for fine-tuned coordination of pixels on the OLED! Pretty neat.

V. Methods

Now, we detail the execution of the four checkoffs for Lab 2, and the how & why of arriving at that execution.

A. Implement OLED Interface Using SPI

The first order of business is to configure the pins for writing the required code. We used the following pin arrangement:

TABLE I
CHECKOFF 1 PINMUX CONFIG

OLED Signal	MCU Interface	Physical Pin
MOSI (SI)	SPI_DOUT (MOSI)	7
SCK (CL)	SPI_CLK (SCLK)	5
DC	GPIO	64
Reset (R)	GPIO	1
OLEDCS (CS)	GPIO	2
Vin (+)	3.3 V	VCC
GND (G)	GND	GND

Connecting from the Physical MCU pins to the OLED pins (the signals indicated) gets us set up. Then, we can focus on the programming. After initializing the board, the first order of business is to configure the SPI with `SPIconfig()`.

```
1  void SPIconfig()
2  {
3
4      //
5      // Reset SPI
6      //
7      MAP_SPIReset(GSPI_BASE);
8
9      //
10     // Configure SPI interface
11     //
12     // Using sub Mode 3; using configured CS
13     MAP_SPIConfigSetExpClk(GSPI_BASE,
14                            RCMPeripheralClockGet(PRCM_GSPI),
15             SPI_IF_BIT_RATE,SPI_MODE_MASTER,
16                            (SPI_SUB_MODE_3,
17             I_SW_CTRL_CS |
18             SPI_4PIN_MODE |
19             SPI_TURBO_OFF |
20             SPI_CS_ACTIVEHIGH |
21                            SPI_WL_8));
22
23     //
24     // Enable SPI for communication
25     //
26     MAP_SPIEnable(GSPI_BASE);
27
28     // enable internal SPI CS to satisfy SPI API
29         // using GPIOP CS instead,
30         // to work with writeCommand() and writeData()
31     MAP_SPICSEnable(GSPI_BASE);
32
33 }
```

A few specifics about this SPI configuration: we use `SPI_SUB_MODE_3` (line 16) to ignore the default chip select and ensure we are using the chip select defined in Table I. We also enable the interna SPI pin select, which is defined at pin 50 (line 31). This ensures that the internal SPI API plays nice; we found that if it was not enabled, even though it remains unused, functions like `MAP_SPITransfer()` will hang up and never return from their calls, probably because these SPI read/write functions are expecting Pin 50 to be enabled. If we enable it behind the scenes, we can proceed with our CS config at Pin 2.

From there, we call `Adafruit_Init()` to supply start-up commands to the OLED and turn it on. This mostly consists of calling `writeCommand()` and `writeData()`, which we are tasked with developing. The relevant code is below:

```
#define DC_PIN        0x2    // GPIO_PIN_1 on GPIOA1_BASE
#define RESET_PIN     0x4    // GPIO_PIN_2 on GPIOA1_BASE
#define OLED_CS_PIN   0x8    // GPIO_PIN_3 on GPIOA1_BASE

// Macros for controlling pins (short form)
#define DC_COMMAND()      GPIOPinWrite(GPIOA1_BASE, DC_PIN, 0)
// DC = LOW for commands
#define DC_DATA()         GPIOPinWrite(GPIOA1_BASE, DC_PIN, DC_PIN)
// DC = HIGH for data
#define CS_ENABLE()       GPIOPinWrite(GPIOA1_BASE, OLED_CS_PIN, 0)
// CS = LOW to enable
#define CS_DISABLE()      GPIOPinWrite(GPIOA1_BASE,        // CS = HIGH to disable
                                       OLED_CS_PIN,
                                       ED_CS_PIN)

//*****************************************************************************
void writeCommand(unsigned char c) {
    // dummy char to satisfy MISO line
    unsigned char dummy;

    // Data Command Pin should be low for commands, CS is low to enable
    DC_COMMAND();
    CS_ENABLE(); // Enable LOW

    // 0 disables automatic CS control; use manual GPIO CS
    MAP_SPITransfer(GSPI_BASE, &c, &dummy, 1, 0);

    CS_DISABLE(); // Disable HIGH
}

//*****************************************************************************
// See writeCommand for explanatory comments
void writeData(unsigned char c) {
    // dummy char to satisfy MISO line
    unsigned char dummy;

    // DC is set to high for Data
    DC_DATA();
    CS_ENABLE();

    // transfer data from SPI line
    MAP_SPITransfer(GSPI_BASE, &c, &dummy, 1, 0);

    // disable chip select when transfer is complete
    CS_DISABLE();
}
```

For lines 1-11, we have defined some simple macros to make the two functions more readable. In short: `DC_COMMAND()` sets the DC pin to low, `DC_DATA()` sets the DC pin to high, `CS_ENABLE()` enables our chip select, and `CS_DISABLE()` disables our chip select. With this in mind, `writeCommand()`:

1) Sets the DC to low (for Command type to the MOSI line)
2) Enables CS to low (indicating data is being transmitted)
3) Sends the `char c` via SPI across the MOSI line
4) Disables CS to high (indicating data is no longer being transmitted).

`writeData()` is similar:

1) Sets the DC to high (for Data type to the MOSI line)
2) Enables CS to low
3) Sends the `char c` via SPI across the MOSI line
4) Disables CS to high

This specific protocol respects SPI and write commands and data to the OLED. This is used extensively for preparing to turn on the OLED and draw pixels on it's display. Fortunately the API takes care of the rest of the implementation details, and we need not concern ourself with the internals for this report.

We can now focus on implementing the OLED output. After it is initialized, we use the following logic to complete the checkoff:

- First, printing the full character set as defined in `glcdfont.h`:

```
// Print full character set
int i = 0; int x = 0; int y = 0;
bool finished = false;
for (x = 0; x < SSD1351WIDTH && !finished; x += PIXEL_WIDTH) {
    for (y = 0; y < SSD1351HEIGHT; y += PIXEL_WIDTH) {
        if (i < GLCD_FONT_SIZE) {
            drawChar(x, y, font[i++], 1, 1, 1);
        }
        else {
            finished = true;
            break;
        }

    }
}
```

We loop through the entire ASCII font array, using `PIXEL_WIDTH` in our iterations to ensure the characters are sufficiently large spaced out (otherwise they would overlap). `drawChar()` is sufficient for our needs to draw each of these, and that is about the extent of this implementation.

- "Hello, world!" is pretty similar, and simpler. We just need to account for the null termination to end the looping.

```
// Print hello world
unsigned char greeting[] = "Hello world!";
i = 0; x = 0; y = 0;
while (greeting[i] != '\0') {
    drawChar(x, y, greeting[i], 1, 1, 1);
    x += PIXEL_WIDTH;
    i++;
}
```

- Printing the 8 bands of different colors involves using a `colors[]` array (colors are 6-digit hexadecimal, interpreted RGB from left to right in their respective [0,255]

range). We can then invoke the width and height of the OLED to evenly partition 8 bands, and draw the lines at each partition, assigning a different color to each. `drawFastHLine()` works for each horizontal line in an iterative loop, and `drawFastVLine()`is good for the vertical lines. We increment the respective axis for each by gaps of `SSD1351HEIGH / num_band`, and hold the other constant, achieving the intended functionality.

- In between each drawing, we call `UtilsDelay(DRAW_DELAY)`followed by `fillScreen(WHITE)`, which allows for an approximate 1.5 second interval in between each drawing, and the screen to refresh to white in preparation for the next one (otherwise all of these drawings will overlap!). The remaining checkoff drawings are drawn; these are pretty self-explanatory, since they are already implemented for us. Checkoff 1 is complete!

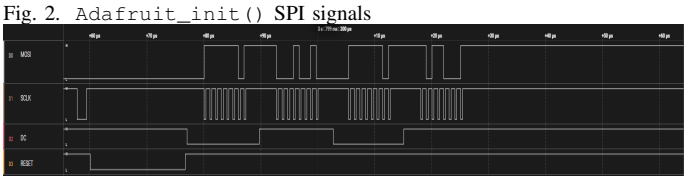
B. Verifying SPI waveforms using a Saleae logic

Now comes the more engaging part. By plugging in the Saleae Logic Analyzer to select pins, we can see what their signals are. We did this at the very start of the `Adafruit_init()` for a few seconds, which looks like so...

```
void Adafruit_Init(void) {


  volatile unsigned long delay;

  // Using GPIO Pin 01 for RESET
  GPIOPinWrite(GPIOA1_BASE, 0x4, 0);      // RESET = RESET_LOW

  UtilsDelay(80000);

  GPIOPinWrite(GPIOA1_BASE, 0x4, 0x4);   // RESET = RESET_HIGH
  UtilsDelay(80000);

        // Initialization Sequence
  writeCommand(SSD1351_CMD_COMMANDLOCK);   // set command lock
  writeData(0x12);
  writeCommand(SSD1351_CMD_COMMANDLOCK);   // set command lock
  writeData(0xB1);

  writeCommand(SSD1351_CMD_DISPLAYOFF);                    // 0xAE
```

With this in mind, we should expect to see the values from the `writeCommand()` and `writeData()` calls be reflected in our signals. Let's look:

Fig. 2. `Adafruit_init()` SPI signals



Here, we see 4 bytes being read from the MOSI line. Each "clock burst" (when the serial clock line is oscillating) is when data is being read; each bit for each byte is read on the rising edge of the clock. It is also apparent this is a byte, since each burst consists of 8 rising edges. The `DC` is low when it is a `writeCommand()`, and high when it is is a `dataCommand()`, which aligns with how the OLED works. Lastly, the reset, is toggled low, then high; at this point in time, the Adafruit is alive and interpreting received commands. We don't see the chip select here (since we only have 4 channels to work wtih), but it looks good too: it is high when there is writes to perform, and low when there is not. The following table breaks down the value we're receiving for each clock burst, mapped to each respective write function:

TABLE II
FIRST 4 WRITE CALL VALUES IN `ADAFRUIT_INIT()`

Command	Argument	Binary
`writeCommand(SSD1351_CMD_COMMANDLOCK)`	0xFD	11111101
`writeData(0x12)`	0x12	00010010
`writeCommand(SSD1351_CMD_COMMANDLOCK)`	0xFD	11111101
`writeData(0xB1)`	0xB1	10110001

Breaking down the binary, if we compare these values (from top to bottom) to the clock bursts (from left to right), we can see they are identical. This is how the SPI protocol is relaying data to the OLED. The other signals, as mentioned, instigate how to read the bits, and when it is a command byte or a data byte.

C. Controlling the BMA222 accelerometer with I2C

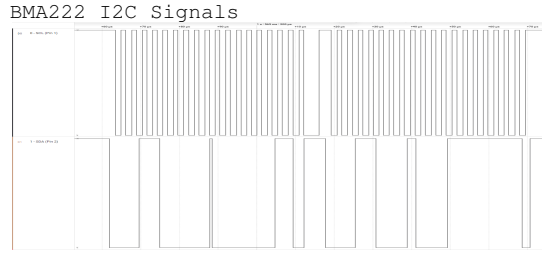
In this checkoff, we use `i2c_demo` to make sure that we can communicate with the accelerometer, and determine if we are getting good tilt values. We don't write any code, we simply run some commands in the UART console to inspect the values and verify it against our Saleae signals. The two commands we issued are as follows:

1) `readreg 0x18 0x3 1`
2) `readreg 0x18 0x5 1`

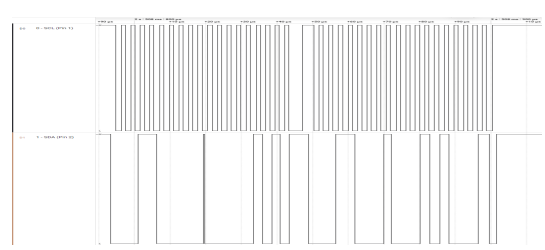
`0x18` is the I2C device address reserved for the accelerometer. `0x3` is the register at this address where the current x-axis coordinate resides. `0x5` is where the y-axis coordinate resides. The `1` is the read length, indicating we extract 1 byte (since the second argument is an offset, and we could extend further if we wanted to).

Now, let's see what our I2C signals look like when we run these in the UART terminal.

Fig. 3. `BMA222 I2C Signals`



(a) X-Axis Signal



(b) Y-Axis Signal

The table below is a break down of what the SDA line is transmitting. As expected, the bits are read on the rising edge of the SCL clock:

TABLE III
READREG VALUES

	Device Address (Write)	Register Pointer	Device Address (Read)	Data Byte
Hex	0x30	0x03	0x31	0x3F
Binary	00110000	00000011	00110001	00111111

(a) X-Axis

	Device Address (Write)	Register Pointer	Device Address (Read)	Data Byte
Hex	0x30	0x05	0x31	0x28
Binary	00110000	00000101	00110001	00101000

(b) Y-Axis

So what's happening here? The device address is `0x18`, our original address, shifted to the left 1 bit. Why? It is actually 7-bit representation due to it actually being signed, so on the line, an extra 8th bit is added at the end. This is a `0` if it is a write, and a `1` if it is a read. The register pointer is where we are reading from, which corresponds to the x and y axis register. Finally, the data byte is what was actually read.

If we piece this together and read the binary from left to right across the SDA, we can see it is accurate. The data bytes themselves, when converted to decimal, are -63 for X and 40 for Y. The sign indicates the angle on a given plane, and the value the magnitude. X is likely side-to-side, and Y forward-and-backward (when looking at the MCU from a top-down perspective, longways). This is pretty helpful for our next checkoff!

D. Sliding Ball

Everything comes to together to make pixels move on the screen with the magic of microcontrollers. First, our pins for this implementation, which requires some changes since we have reserved pins for I2C (for the accelerometer):

TABLE IV
CHECKOFF 4 PINMUX CONFIG

OLED Signal	MCU Interface	Physical Pin
MOSI (SI)	SPI_DOUT (MOSI)	7
SCK (CL)	SPI_CLK (SCLK)	5
DC	GPIO	64
Reset (R)	GPIO	3
OLEDCS (CS)	GPIO	4
N/A	I2C_SCL	1
N/A	I2C_SDA	2
Vin (+)	3.3 V	VCC
GND (G)	GND	Anf that concluGND

After shifting around some pins, we're ready to get started. All of the SPI configuration and OLED initialization is effectively the same, with the exception of changing the pin values being used (for Reset and OLEDCS). We also need to initialize I2C, which is simply calling `I2C_IF_Open(I2C_MASTER_MODE_FST)`. Now, the code which makes the pitching ball is below. Let's walk through it:

```
1  // registers for Accelerometer values (I2C SDA and SLA respectively)
2  unsigned char reg_x = 0x03, reg_y = 0x05;
3  // int8_t to respect 2's complement
4  int8_t x_raw, y_raw;
5
6  // curb acceleration speed
7  float speed = 0.1;
8
9  // Initial position: center of screen
10 int x = SSD1351WIDTH / 2, y = SSD1351HEIGHT / 2;
11 // achieves a radius of about 2 pixels
12 int radius = 2;
13 // Run accelerometer circle on Adafruit OLED in a loop
14 fillScreen(WHITE);
15 fillCircle(x, y, radius, RED);
16 while(1)
17 {
18     // refresh x,y variables based on accelerometer
19             // read registers, then write to var
20     I2C_IF_Write(0x18, &reg_x, 1, 0);
21     I2C_IF_Read(0x18, (unsigned char *)&x_raw, 1);
22
23     I2C_IF_Write(0x18, &reg_y, 1, 0);
24     I2C_IF_Read(0x18, (unsigned char *)&y_raw, 1);
25
26     // Update position
27     int next_x = x + (int)(x_raw * speed);
28     int next_y = y + (int)(y_raw * speed);
29
30     // Bound to screen edges
31     if(next_x < radius) next_x = radius;
32     if(next_x > SSD1351WIDTH - radius) next_x = SSD1351WIDTH - radius;
33     if(next_y < radius) next_y = radius;
34     if(next_y > SSD1351HEIGHT - radius) next_y = SSD1351HEIGHT - radius;
35
36     // redraw if ball needs to move
37     if (next_x != x || next_y != y) {
38         // erase old ball
39         fillCircle(x, y, radius, WHITE);
40
41         // Update to new coordinates
42         x = next_x;
43         y = next_y;
44
45         // new ball
46         fillCircle(x, y, radius, RED);
47     }
48
49     // introduce slight delay for less jitter
50     UtilsDelay(80000);
51
52 }
53
54 }
```

1) To keep the ball moving, we enclose the main logic in a while loop. But on line 15, we start the position of the ball in the very center of the screen.
2) Upon entering the loop, we write to the I2C registers asociaed with our X and Y value, then we read from it. This gives us the new tilt values in 2D space (we are ignoring the Z axis since the OLED is a flat screen).
3) With this value, we update the position, the new value is the current value plus the raw value extracted from the I2C registers; we multiply the raw value with a float constant to carefully control the speed of movement (smaller scalar value means less change!), on Lines 27 and 28.
4) These new values are then compared against the width and height of the screen, on Lines 31-34. This ensures the ball is not allowed to go "off" the screen (more accurately, experience undefined behavior in the ball drawing function).
5) Lastly, we check if the ball needs to move on Line 37. If the new x or y differs from the previous x or y, we start by filling the pixels with white (where the ball used to be). We update the coordinates, then we render a new ball at these coordinates. The illuson of movement is complete!
6) Last, we call a delay command to have slightly less jitter; we don't want the input to be too responsive between iterations.

And that concludes checkoff 4!

VI. DISCUSSION

We experienced several difficulties with this lab (hence why it was submitted one lab period late). I will break down the

most notable ones, and try to offer an accounting of our Lab 2 experience.

1) CCS project issues: Both of us experienced issues with building CCS projects correctly. Our linked resources were consistently broken and we were having trouble porting the progress from previous lab parts to current parts. It turns out, we were approaching the CCS project environment wrong: moving files outside of the operations of the CCS environment was not respecting changes to the path environment variables. Once we started manipulating the files in the CCS IDE, these problems were smoothed over. We also attempted to create several workspaces, but the organizational hierarchy of our folders left something to be desired, and it consistently confused the IDE. Once we opted for one workspace, with all projects at the root folder, these problems also disappeared. Our attempts to be organized and respect good GitHub etiquette culminated into these issues. Once we played nice with Texas Instruments software, there weren't any other significant issues here.
2) Hardware issues: A couple of times, we did not provision the right equipment for at-home work (mainly the connector pins). This ended up causing us to be late as well. We've configured out environment to ensure this does not happen again.
3) SPI API subtleties: This was a serious bottleneck. The SPI API for speaking with Adafruit is rather confusing. Specifically, it has an internal Chip Select (discussed earlier in the report) at Pin 50, which conflicts with our manually set OLEDCS chip select at general-purpose Pin 02. Not enabling the internal CS, despite not really using it, caused us a lot of grief; our SPI functions were consistently stalling without doing this. Once we included it in the SPI configuration function, these problems disappeared, but it took longer than we'd care to admit to track the source of this issue. There were other SPI subtleties that ate up time, but this was the main one.
4) Dependency subtleties: There were some dependencies that we had trouble working with. Demos like those provided for I2C and SPI have a lot of code to make it work, and if you choose to leverage that code for your own CCS project and aren't careful, it is easy to miss a couple lines (if you are trying to input the "glue code" manually). We were missing an I2C initialization in `main.c` for Checkoff 4 that had us scratching our heads for awhile. In the future, we'll definitely be more observant when it comes to transporting dependency code.
5) The biggest change that one of us decided on moving forward is in keeping with keep one workspace, they would have folders of different files needed for different parts separately, and when they needed it to run within CCS, they would just use the original SPI demo, gut its contents, and then run since that was the only reasonable way forward.
6) Since both of us also have USB-C only laptops, this further hindered our ability to work quickly due to logistical errors in forgetting cables sometimes as well. Being also on a Mac and Windows laptops separately, there were some issues as well that were caused by having different instructions/ pathways for different machines, and not being able to help each other troubleshooting as effectively.
7) Using Salae as recommended by our TA definitely helped our understanding and fixing of the issue, and we will proceed to keep using it moving
8) An improvement was to start utilizing the demos first and understanding how they worked before going head long into making our own code function.

VII. Contributions

A. Jacob Feenstra

My contributions are as follows:

1) Wrote the Part I code. Troubleshooted SPI API and getting the `Adafruit_init()` to turn on. Eventually succeeded. Wrote core logic for Checkoff I deliverable.
2) Documented the Saleae signal output for Part II.
3) Tested the I2C protocol with the accelerometer data for Part III, and documented the signal output.
4) Wrote the Part IV code. Adjust core logic for moving ball, respecting 2's complement and screen boundary checks. Tweaked to ensure consistent output movement.
5) Wrote the first draft for the report and prepared it for IEEE format.

B. Chun Ho Chen

1) Troubleshooted issues that stemmed from making the code work, and testing/verified that the SPI demo could communicate across both devices
2) Troubleshooted the SPI and oled code not working, and trying different pieces of code to get it working again.
3) Ran the I2C Demo, and verified outputs
4) Wrote some of the code to initialize the ball onto the screen
5) Edited and finalized the report draft